

ROS NEDİR VE NASIL ÇALIŞIR?

Robot İşletim Sistemi (ROS) = robotik araştırma ve endüstrisinde yaygın olarak kullanılan açık kaynaklı bir meta-işletim sistemi ve yazılım çerçevesidir. Geleneksel bir işletim sisteminin (Linux gibi) üzerinde çalışır ve geliştiricilere robot uygulamaları oluşturmak, sensörleri koordine etmek ve karmaşık görevleri yönetmek için gerekli olan kütüphane, araç ve iletişim altyapısı koleksiyonunu sağlar. ROS un en temel gücü çok çeşitli robotik platformlarla çalışacak şekilde tasarlanmış olmasıdır bu da onu robot uzmanları için esnek güçlü ve tekrar kullanılabilir bir standart haline getirir.

ROS robotun görevlerini tek bir büyük program yerine dağıtık ve bağımsız yazılım bileşenleri halinde çalıştırarak işlev görür.

Düğüm (Nodes) = Robotun her bir görevi ayrı bir küçük program (düğüm) olarak çalışır bu sayede bir düğüm çökse bile tüm sistem çalışmaya devam edebilir.

ROS Master: Ağdaki tüm düğümlerin kaydını tutan ve onların birbirlerini bulmasını sağlayan merkezi bir rehberlik mekanizmasıdır.

İletişim (Topics) = Düğümler, verileri belirli kanallar üzerinden asenkron ve tek yönlü olarak iletir.

- Yayıncı (Publisher) = Veriyi üreten düğüm ilgili konuya mesaj yayınlar.
- Abone (Subscriber) = Veriye ihtiyacı olan düğüm o konuya abone olur ve gelen mesajları işler.

Hizmetler (Services) = Anlık istek cevap gerektiren işlemler için kullanılır. Bir düğüm istek gönderir, diğeri bu isteği işler ve sonucu geri döndürür

Araçlar (tools) = ROS bu modüler sistemin geliştirilmesi test edilmesi ve görselleştirilmesi için bir dizi standart araç seti sağlar.

ROS 1 ve ROS 2 arasındaki temel farklar nelerdir?

ROS1, robotik araştırmalarını hızlandırmak amacıyla 2007 yılında [Willow Garage](#) tarafından geliştirildi ve piyasaya sürüldü o zamandan beri ROS1 robotik topluluğunda büyük bir popülerlik kazandı ve araştırma ve deney yapmak için fiili platform haline geldi ancak ROS1 ticari kullanım düşünülerek geliştirilmedi bu nedenle güvenlik ağ topolojisi ve sistem çalışma süresi gibi konulara öncelik verilmedi. Dolayısıyla ROS un ticari alanda benimsenmesiyle birlikte, birçok önemli kusuru daha da belirgin hale geldi. Bu nedenle, ROS u ticari kullanım düşünülerek sıfırdan yeniden inşa etme ihtiyacı doğdu yani ROS2.

ROS ilk sürümü olan ROS 1, robotik arařtırmalar için ıır aan bir temel atmıř olsa da, modern endüstriyel talepler doėrultusunda ROS 2 adıyla tamamen yeniden tasarlanmıřtır. Bu iki sürüm arasındaki farklılıklar yalnızca küçük güncellemeler deėil, robotik sistemlerin temel alıřma prensiplerini kökten deėiřtiren mimari kararlardır.

1. İletişim Altyapısındaki Devrim: Merkeziyetten Daėıtıklıėa Geiř

ROS 1 iletişimi yönetmek için RoSMaster adı verilen merkezi bir sunucuya (Master Node) baėlıydı. Bu sistem, tüm düėümünün birbirini bulması için zorunluydu. Ancak bu durum, sistem için tek bir hata noktası yaratıyordu: Master ökerse, tüm robot sistemi duruyordu.

Buna karřılık, ROS 2 tamamen daėıtık bir yapıya geiř yapmıřtır. İletişim altyapısı olarak DDS (Daėıtılmıř Veri Daėıtım Hizmeti) adı verilen endüstriyel bir standardı benimser. Bu sayede, merkezi bir Master düėümüne ihtiya kalmaz; düėümler birbirini otomatik olarak bulur ve doėrudan iletişim kurar.

2. Güvenlik ve Gerek Zamanlılık

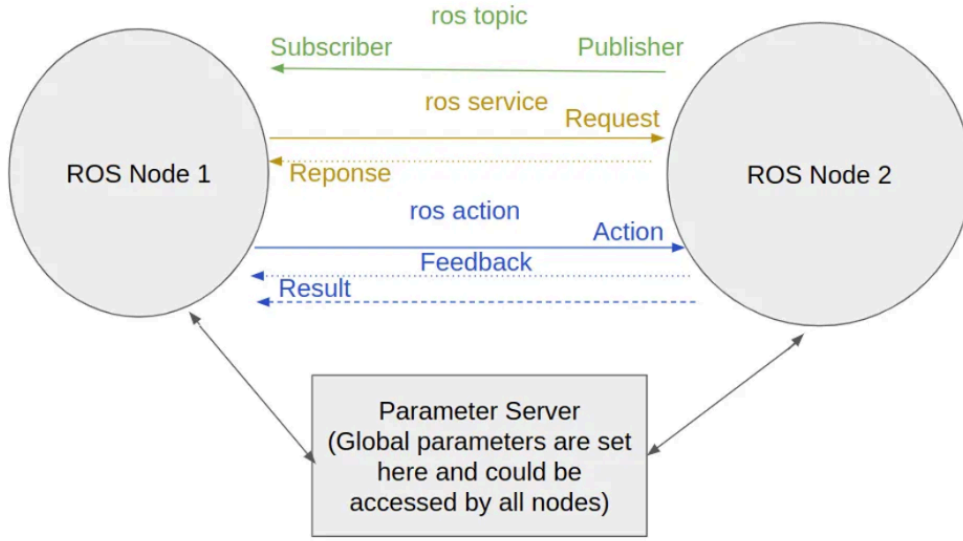
ROS 1 akademik laboratuvar ortamlarında geliřtirildiėi için güvenlik ve gerek zamanlılık konularına öncelik vermiyordu aė iletişimi varsayılan olarak řifreli deėildi ve sistem hassas zamanlamalar gerektiren uygulamalar için uygun deėildi.

ROS 2 ise endüstriyel standartları gözeterek bu eksiklikleri gidermiřtir. DDS kullanımı, iletişime yerleřik güvenlik katmanları řifreleme, kimlik doėrulama eklemiřtir. En önemlisi, gerek zamanlı (Real-Time) iřletim sistemleriyle uyumlu alıřabilme yeteneėi kazanmıřtır bu ROS 2 nin bir robotun motorlarını veya kritik sensörlerini önceden belirlenmiř hassas zaman dilimleri içinde kontrol edebileceėi anlamına gelir bu da otonom aralar ve cerrahi robotlar gibi kritik uygulamalar için hayati önem tařır.

3. Platform ve Geliřtirme Ortamı Esnekliėi

ROS 1 büyük ölçüde yalnızca Ubuntu Linux üzerinde alıřacak řekilde tasarlanmıřtı. Derleme sistemi olarak Catkin kullanılıyordu.

ROS 2 bu sınırlamayı kaldırarak Linuxun yanı sıra Windows macOS ve hatta gömülü sistemler gibi ok eřitli platformları destekler hale gelmiřtir. Derleme sistemi de daha modern ve esnek bir ara olan Ament/Colcon ile deėiřtirilmiřtir. Bu geniř platform desteėi, geliřtiricilere daha fazla özgürlük tanıımıř ve robotik yazılımların ticari ürünlere entegrasyonunu kolaylařtırmıřtır.



ROS'un robotik projelerinde yaygın kullanım alanları nelerdir?

- Otonom araçlar ve dronlar
- Endüstriyel robotik
- İnsansı robotlar
- Tıbbi robotik
- Tarım robotları
- İnsansız hava araçları (İHA'lar)
- Servis robotları
- Araştırma ve eğitim

ROS'un otonom araba projelerindeki rolü nedir?

ROS otonom bir arabanın bütün parçalarını konuşuran, organize eden senkronize eden merkezi sinir sistemi gibidir. Yani arabaya gör düşün karar ver hareket et demeyi sağlayan yazılımsal alt yapıyı oluşturur

Sensör Yönetimi =

Lidar, radar, kamera, GPS, IMU vb. sensörden akan veriyi toplar işler ve kullanıma hazır bir formata dönüştürür.

Konumlama ve Haritalama (SLAM) =

Araç kendi bulunduğu konumu belirlemek ve çevresinin haritasını oluşturmak zorundadır. ROS, bunu yapan hazır algoritmaları paket olarak sunar.

Algılama ve Çevre Analizi =

ROS üzerindeki paketler kamera görüntüsü veya nokta bulutundan yayaları araçları yolları şeritleri engelleri tespit edebilir.

Yol Planlama =

Araç nereye gidecek nasıl gidecek, hangi yol daha güvenli hangi rota daha kısa bunları hesaplayan planlama algoritmaları ROS üzerinde hazırdir

Karar Verme ve Kontrol =

Araçın hızını, direksiyon açısını frenlemeyi ve genel davranışını kontrol eden modüller ROS üzerinde yönetilir.

Modüler ve Dağıtık Çalışma =

Farklı bilgisayarlar tek bir robot gibi çalışabilir bu otonom araçlarda çok önemlidir çünkü sensör işleme karar verme kontrol gibi ağır işler paralel yürütülür.

Açık Kaynak Ekosistemi =

Zaten yapılmış binlerce paket sayesinde sıfırdan sistem yazmana gerek kalmaz.

Güçlü Görselleştirme / Kayıt Araçları =

RViz rqt rosbag gibi araçlar sayesinde hataları görmek sensör verisini izlemek davranışları analiz etmek çok kolaydır geliştirme süresi ciddi şekilde hızlanır.

ROS 2 JAZZY

Ortamı Yapılandırma

= ortamın ros 2 jazzy ile çalıştığını bilmesi için yapıyoruz otomatik ve manuel olarak

```
omer@omer-Sword-16-HX-B14VGKG:~$ echo $ROS_DISTRO
jazzy
omer@omer-Sword-16-HX-B14VGKG:~$ source /opt/ros/jazzy/setup.bash
omer@omer-Sword-16-HX-B14VGKG:~$ echo $ROS_DISTRO
jazzy
omer@omer-Sword-16-HX-B14VGKG:~$ ros2 --help

usage: ros2 [-h] [--use-python-default-buffering]
           Call 'ros2 <command> -h' for more detailed usage. ...

ros2 is an extensible command-line tool for ROS 2.

options:
  -h, --help                show this help message and exit
  --use-python-default-buffering
                           Do not force line buffering in stdout and instead use
                           the python default buffering, which might be affected
                           by PYTHONUNBUFFERED/-u and depends on whatever stdout
                           is interactive or not

Commands:
  action      Various action related sub-commands
  bag         Various rosbag related sub-commands
  component   Various component related sub-commands
  daemon      Various daemon related sub-commands
  doctor      Check ROS setup and other potential issues
  interface   Show information about ROS interfaces
  launch      Run a launch file
  lifecycle   Various lifecycle related sub-commands
  multicast   Various multicast related sub-commands
  node        Various node related sub-commands
  param       Various param related sub-commands
  pkg         Various package related sub-commands
  run         Run a package specific executable
  security    Various security related sub-commands
  service     Various service related sub-commands
  topic       Various topic related sub-commands
  wtf         Use 'wtf' as alias to 'doctor'

Call 'ros2 <command> -h' for more detailed usage.
omer@omer-Sword-16-HX-B14VGKG:~$ omer@omer-Sword-16-HX-B14VGKG:~$ ros2 --help
```

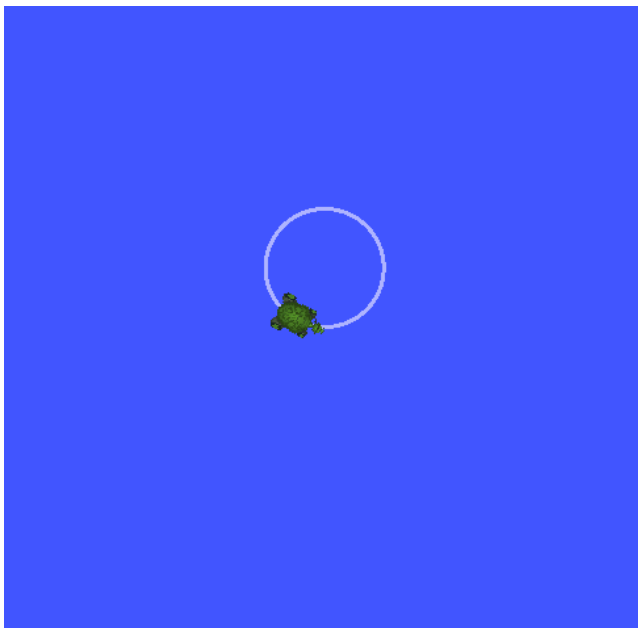
ros2 help koduyla da ros2 imin doğru çalışıp çalışmadığına bakıyorum

Using Turtlesim ROS2 ve RGT

Turtlesim

turtlesim deneyiminde Ubuntu terminalimden kaynaklı ok tuşlarıyla kaplumbağayı doğrudan kontrol edemedim bu nedenle çalışma ortamımı hazırladıktan sonra bir kaplumbağa görsel simülasyonu oluşturdum sonrasında başka bir terminal üzerinden mesajlar göndererek kaplumbağaya yön verip spiral hareket yapmasını sağladım

```
omer@omer-Sword-16-HX-B14VGKG:~$ source /opt/ros/jazzy/setup.bash
omer@omer-Sword-16-HX-B14VGKG:~$ ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 1.0}, angular: {z: 1.0}}"
publisher: beginning loop
publishing #1: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=1.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=1.0))
publishing #2: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=1.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=1.0))
publishing #3: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=1.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=1.0))
```



```

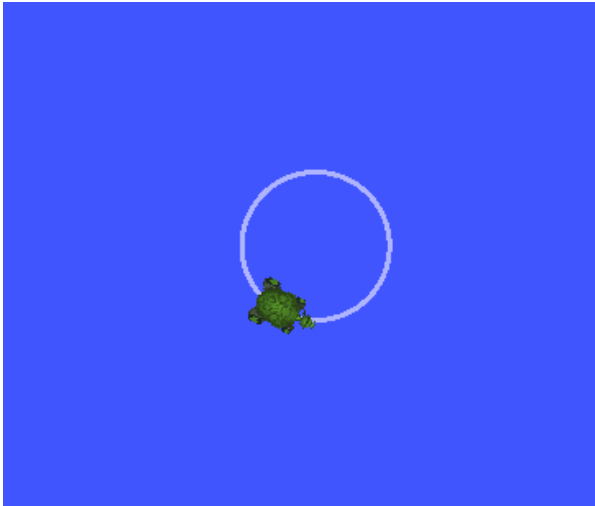
omer@omer-Sword-16-HX-B14VGKG:~$ source /opt/ros/jazzy/setup.bash
omer@omer-Sword-16-HX-B14VGKG:~$ ros2 run turtlesim turtlesim_node
[INFO] [1765662879.631922390] [turtlesim]: Starting turtlesim with node name /turtlesim
[INFO] [1765662879.634421813] [turtlesim]: Spawning turtle [turtle1] at x=[5.544445], y=[5.544445], theta=[0.000000]

```

RGT

çalışma ortamımı hazırladıktan sonra rqt aracı kullanarak kaplumbağanın hareket ettiği topicleri izledim başka bir terminalde turtle1/cmd_vel üzerinden yayınlanan Twist mesajları ile kaplumbağanın ileri hareket ederek spiral çizmesi sağlandı ve rqt üzerinden bu hareketlerin mesajlarını canlı olarak gözlemledim.

File Plugins Running Perspectives Help				
Topic Monitor				
Topic	Type	Bandwidth	Hz	Value
☐ /parameter_events	rcl_interfaces/msg/ParameterEvent			not monitored
☐ /rosout	rcl_interfaces/msg/Log			not monitored
☒ /turtle1/cmd_vel	geometry_msgs/msg/Twist	51.65B/s	0.98	
linear	geometry_msgs/Vector3			
x	double			1.0
y	double			0.0
z	double			0.0
angular	geometry_msgs/Vector3			
x	double			0.0
y	double			0.0
z	double			0.5
☒ /turtle1/color_sensor	turtlesim/msg/Color	74.23B/s	9.28	
r	uint8			179
g	uint8			184
b	uint8			255
☒ /turtle1/pose	turtlesim/msg/Pose	222.68B/s	9.28	
x	float			3.664191246032715
y	float			8.247774124145508
theta	float			-1.9341500997543335
linear_velocity	float			1.0
angular_velocity	float			0.5
☐ /turtle1/rotate_absolute/_action/feedback	turtlesim/action/RotateAbsolute_FeedbackMessage			can not get message class
☐ /turtle1/rotate_absolute/_action/status	action_msgs/msg/GoalStatusArray			not monitored



NODE ları Anlamak

Subscribers → turtlesim node unun dinlediği topicler

`/turtle1/cmd_vel`: `geometry_msgs/msg/Twist` → kaplumbağaya hız komutlarını dinliyor

`/parameter_events` → ROS parametre değişikliklerini dinliyor

Publishers → turtlesim node unun yayınladığı topicler

`/turtle1/pose`: `turtlesim/msg/Pose` → kaplumbağanın gerçek konum ve yön bilgisi

`/turtle1/color_sensor` → renk bilgisi

`/rosout` → log mesajları

Service Servers → node un sunduğu servisler

Örnek: `/clear`, `/kill`, `/reset`, `/spawn`, `/turtle1/set_pen`

Action Servers → node un sunduğu aksiyonlar

`/turtle1/rotate_absolute` → kaplumbağayı belirli bir açıda döndürmek için

Topic ilişkisi

`/turtle1/cmd_vel` → Publisher sen olacaksın (mesaj gönderiyorsun)

`/turtle1/pose` → Subscriber turtlesim node'u (mesajları alıp konum güncelliyor)

Temel gözlem

`/turtle1/cmd_vel` ile Twist mesajı yayınla

/turtle1/pose topic'ini rqt veya `ros2 topic echo /turtle1/pose` ile izle böylece mesaj gönderdiğinde kaplumbağanın pozisyonunun değiştiğini görüyorsun İleri düzey servis ve actionlar

Sen (publisher) → /turtle1/cmd_vel → turtlesim node (subscriber) → /turtle1/pose (publish) → sen veya rqt ile gözlem

TOPİCLER

topicler ros 2 de node lar arasında veri alışverişi yapılmasını sağlayan temel iletişim yapısıdır bir node bir topic üzerinden veri yayınlarken başka bir node aynı topic i dinleyerek bu veriyi alır bu çalışmada turtlesim uygulamasında kullanılan turtle1 cmd_vel topicini incelenmiştir yazdığım publisher node bu topic üzerinden geometry_msgs twist mesajları yayınlamıştır yayınlanan bu mesajlar kaplumbağanın hız ve yön bilgisini belirlemiş ve kaplumbağanın ekranda hareket etmesini sağlamıştır ros2 topic list ve ros2 topic echo komutları kullanılarak aktif topicler ve bu topicler üzerinden akan veriler terminal üzerinden gözlemlenmiştir

```
omer@omer-Sword-16-HX-B14VGKG:~$ ros2 topic list
/parameter_events
/rosout
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
omer@omer-Sword-16-HX-B14VGKG:~$ ros2 topic info /turtle1/pose
Type: turtlesim/msg/Pose
Publisher count: 1
Subscription count: 1
omer@omer-Sword-16-HX-B14VGKG:~$ ros2 topic echo /turtle1/pose
x: 7.479196071624756
y: 8.019560813903809
theta: 1.8066521883010864
linear_velocity: 1.0
angular_velocity: 0.5
---
x: 7.475332736968994
y: 8.035086631774902
theta: 1.8146522045135498
linear_velocity: 1.0
angular_velocity: 0.5
---
x: 7.4713454246521
y: 8.050581932067871
theta: 1.8226522207260132
linear_velocity: 1.0
angular_velocity: 0.5
---
x: 7.4672346115112305
y: 8.066044807434082
theta: 1.8306522369384766
linear_velocity: 1.0
angular_velocity: 0.5
```

```

omer@omer-Sword-16-HX-B14VGKG:~$ ros2 node list
/rqt_gui_py_node_15337
/turtlesim
omer@omer-Sword-16-HX-B14VGKG:~$ ros2 node info /turtlesim
/turtlesim
Subscribers:
  /parameter_events: rcl_interfaces/msg/ParameterEvent
  /turtle1/cmd_vel: geometry_msgs/msg/Twist
Publishers:
  /parameter_events: rcl_interfaces/msg/ParameterEvent
  /rosout: rcl_interfaces/msg/Log
  /turtle1/color_sensor: turtlesim/msg/Color
  /turtle1/pose: turtlesim/msg/Pose
Service Servers:
  /clear: std_srvs/srv/Empty
  /kill: turtlesim/srv/Kill
  /reset: std_srvs/srv/Empty
  /spawn: turtlesim/srv/Spawn
  /turtle1/set_pen: turtlesim/srv/SetPen
  /turtle1/teleport_absolute: turtlesim/srv/TeleportAbsolute
  /turtle1/teleport_relative: turtlesim/srv/TeleportRelative
  /turtlesim/describe_parameters: rcl_interfaces/srv/DescribeParameters
  /turtlesim/get_parameter_types: rcl_interfaces/srv/GetParameterTypes
  /turtlesim/get_parameters: rcl_interfaces/srv/GetParameters
  /turtlesim/get_type_description: type_description_interfaces/srv/GetTypeDescription
  /turtlesim/list_parameters: rcl_interfaces/srv/ListParameters
  /turtlesim/set_parameters: rcl_interfaces/srv/SetParameters
  /turtlesim/set_parameters_atomically: rcl_interfaces/srv/SetParametersAtomically
Service Clients:

Action Servers:
  /turtle1/rotate_absolute: turtlesim/action/RotateAbsolute
Action Clients:

```

Hizmetler (Services)

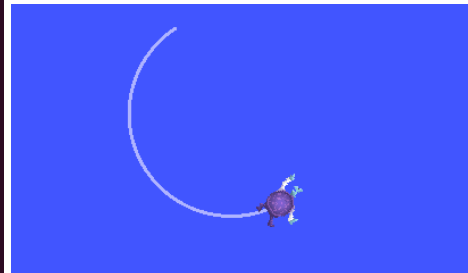
Çalışma ortamımı hazırladıktan sonra /clear servisini kullandım ROS 2 servisleri bir node dan başka bir node a belirli bir işlem yapması için gönderilen talepler olarak çalışıyor /clear servisi turtlesim ekranındaki çizgileri temizlemek için kullanılıyor ve kaplumbağayı hareket ettirirken temiz bir alan sağlamış oldu

```

omer@omer-Sword-16-HX-B14VGKG:~$ ros2 service list
/clear
/kill
/reset
/rqt_gui_py_node_15337/describe_parameters
/rqt_gui_py_node_15337/get_parameter_types
/rqt_gui_py_node_15337/get_parameters
/rqt_gui_py_node_15337/get_type_description
/rqt_gui_py_node_15337/list_parameters
/rqt_gui_py_node_15337/set_parameters
/rqt_gui_py_node_15337/set_parameters_atomically
/spawn
/turtle1/set_pen
/turtle1/teleport_absolute
/turtle1/teleport_relative
/turtlesim/describe_parameters
/turtlesim/get_parameter_types
/turtlesim/get_parameters
/turtlesim/get_type_description
/turtlesim/list_parameters
/turtlesim/set_parameters
/turtlesim/set_parameters_atomically
omer@omer-Sword-16-HX-B14VGKG:~$ ros2 service type /clear
ros2 interface show std_srvs/srv/Empty
std_srvs/srv/Empty
---
omer@omer-Sword-16-HX-B14VGKG:~$ ros2 service call /clear std_srvs/srv/Empty
waiting for service to become available...
requester: making request: std_srvs.srv.Empty_Request()

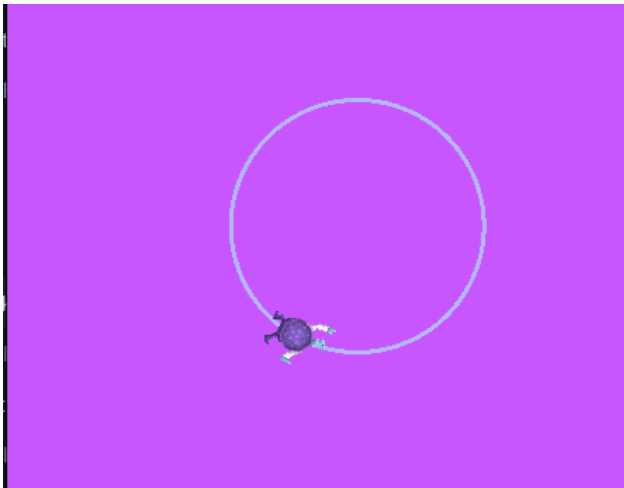
response:
std_srvs.srv.Empty_Response()

```



Parametreler (Parameters)

Çalışma ortamımı hazırladıktan sonra /turtlesim node unun parametrelerini inceledim ROS 2 parametreleri bir node un davranışını değiştirmek için kullanılan ayarlar olarak çalışıyor background_r parametresi turtlesim ekranının kırmızı rengini ayarlamak için kullanılıyor ve değeri değiştirerek simülasyonun arka plan rengini istediğim şekilde güncellemiş oldum



```
omer@omer-Sword-16-HX-B14VGKG:~$ ros2 param list /turtlesim
background_b
background_g
background_r
holonomic
qos_overrides./parameter_events.publisher.depth
qos_overrides./parameter_events.publisher.durability
qos_overrides./parameter_events.publisher.history
qos_overrides./parameter_events.publisher.reliability
start_type_description_service
use_sim_time
omer@omer-Sword-16-HX-B14VGKG:~$ ros2 param get /turtlesim background_r
Integer value is: 69
omer@omer-Sword-16-HX-B14VGKG:~$ ros2 param set /turtlesim background_r 200
Set parameter successful
omer@omer-Sword-16-HX-B14VGKG:~$
```

ACTIONS(Aksiyonlar)

Actionlar ROS 2 içerisinde zaman alan ve adım adım gerçekleşen işlemler için kullanılır bir action hedef goal işlem sırasında geri bildirim feedback ve işlem sonunda sonuç result üretir ayrıca gerekirse iptal edilebilir turtlesim üzerinde rotate_absolute actionu kullanılarak kaplumbağanın belirli bir açıya kontrollü şekilde dönmesi sağlanmıştır bir action toplamda 5 temel parçadan oluşur

Goal Hedef =

Node action sunucusuna yapılmasını istediği işi gönderir

Turtlesim örneğinde kaplumbağanın belirli bir açıya dönmesi hedef olarak gönderilmiştir

Action Server =

Gönderilen hedefi alan ve işlemi gerçekten yapan node dur

Turtlesim içerisinde rotate_absolute action server bu görevi yerine getirir

Feedback Geri Bildirim =

İş devam ederken action sunucusundan gelen anlık durum bilgisidir

Kaplumbağanın dönüş sırasında ne kadar ilerlediği bu aşamada takip edilebilir

Result Sonuç =

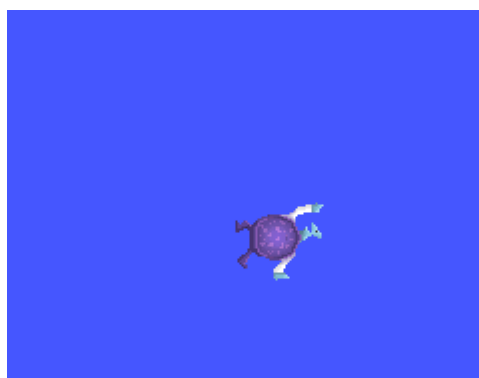
İş tamamlandığında action sunucusundan dönen nihai çıktıdır

Kaplumbağa hedeflenen açıya ulaştığında action başarıyla tamamlanmış olur

Cancel İptal =

İş tamamlanmadan önce gönderilen hedefin durdurulmasını sağlar

Uzun süren işlemlerde kontrol ve güvenlik amacıyla kullanılır



```
omer@omer-Sword-16-HX-B14VGKG:~$ source /opt/ros/jazzy/setup.bash
ros2 action list
/turtle1/rotate_absolute
omer@omer-Sword-16-HX-B14VGKG:~$ ros2 action info /turtle1/rotate_absolute
Action: /turtle1/rotate_absolute
Action clients: 0
Action servers: 1
  /turtlesim
omer@omer-Sword-16-HX-B14VGKG:~$ ros2 topic echo /turtle1/pose
x: 5.544444561004639
y: 5.544444561004639
theta: 1.5520000457763672
linear_velocity: 0.0
angular_velocity: 0.0
---
x: 5.544444561004639
y: 5.544444561004639
theta: 1.5520000457763672
linear_velocity: 0.0
angular_velocity: 0.0
---
x: 5.544444561004639
y: 5.544444561004639
theta: 1.5520000457763672
linear_velocity: 0.0
angular_velocity: 0.0
---
```

```
omer@omer-Sword-16-HX-B14VGKG:~$ source /opt/ros/jazzy/setup.bash
ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute "{theta: 1.57}"
Waiting for an action server to become available...
Sending goal:
  theta: 1.57

Goal accepted with ID: 7681c9dbba574014ac090cf2724a82dd

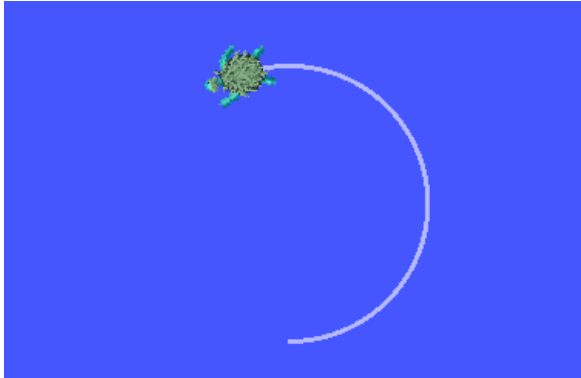
Result:
  delta: -1.5520000457763672

Goal finished with status: SUCCEEDED
omer@omer-Sword-16-HX-B14VGKG:~$
```

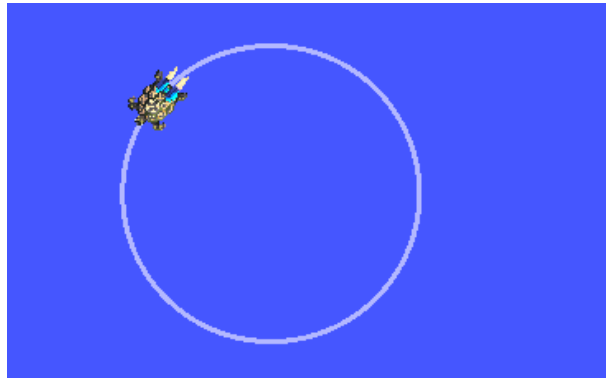
ROSBAG

rosvag kullanılarak turtle1/cmd_vel ve turtle1/pose topiclerinden alınan veriler kaydedilmiş ve daha sonra rosvag play komutu ile bu veriler geri oynatılarak kaplumbağanın aynı hareketleri tekrar etmesi sağlanmıştır

```
omer@omer-Sword-16-HX-B14VGKG:~$ source /opt/ros/jazzy/setup.bash
ros2 bag record /turtle1/cmd_vel /turtle1/pose
[WARN] [ros2bag]: Positional "topics" argument deprecated. Please use optional "--topics" argument instead.
[INFO] [1765702816.800506430] [rosbag2_recorder]: Press SPACE for pausing/resuming
[INFO] [1765702816.805713852] [rosbag2_recorder]: Listening for topics...
[INFO] [1765702816.805755171] [rosbag2_recorder]: Event publisher thread: Starting
[INFO] [1765702816.805757459] [rosbag2_recorder]: Recording...
[INFO] [1765702817.015599282] [rosbag2_recorder]: Subscribed to topic '/turtle1/cmd_vel'
[INFO] [1765702817.019283703] [rosbag2_recorder]: Subscribed to topic '/turtle1/pose'
[INFO] [1765702817.119472556] [rosbag2_recorder]: All requested topics are subscribed. Stopping discovery...
[INFO] [1765702907.719217158] [rosbag2_recorder]: Pausing recording.
[INFO] [1765702907.719266668] [rosbag2_cpp]: Writing remaining messages from cache to the bag. It may take a while
[INFO] [1765702907.720554264] [rosbag2_recorder]: Event publisher thread: Exiting
[INFO] [1765702907.720578840] [rosbag2_recorder]: Recording stopped
omer@omer-Sword-16-HX-B14VGKG:~$ ls
Desktop Documents Downloads İndirilenler Music Pictures Public rosvag2_2025_12_14-12_00_16 snap Templates UnityHub.AppImage Videos wget-log
omer@omer-Sword-16-HX-B14VGKG:~$ source /opt/ros/jazzy/setup.bash
ros2 bag play rosvag2_2025_12_14-12_00_16
[INFO] [1765703132.655366258] [rosbag2_player]: Set rate to 1
[INFO] [1765703132.658781949] [rosbag2_player]: Adding keyboard callbacks.
[INFO] [1765703132.658796613] [rosbag2_player]: Press SPACE for Pause/Resume
[INFO] [1765703132.658799950] [rosbag2_player]: Press CURSOR_RIGHT for Play Next Message
[INFO] [1765703132.658803010] [rosbag2_player]: Press CURSOR_UP for Increase Rate 10%
[INFO] [1765703132.658805625] [rosbag2_player]: Press CURSOR_DOWN for Decrease Rate 10%
[INFO] [1765703132.659995869] [rosbag2_player]: Playback until timestamp: -1
```



ROSBAG İLE KAYIDIN
TEKRAR OYNATILMIŞ HALİ

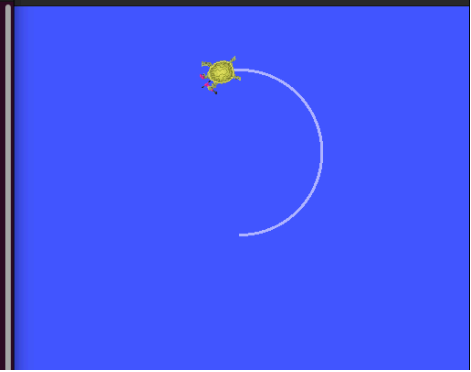


İLK BAŞTA KAYIT ALINAN

- Twist tipinde hız verisini gönderen kodu yazarak test ediniz

Bu aşamada ROS 2 için bir çalışma alanı oluşturdum ve ros2_ws ismini verdim
Bu çalışma alanı altında omer_deneme isimli bir ROS 2 paketi oluşturdum
Paketi Python tabanlı olacak şekilde ament_python build sistemi ile yapılandırdım
Oluşturduğum paket içerisinde turtlesim uygulamasını kontrol etmek amacıyla bir node yazdım bu node geometry_msgs Twist mesaj tipini kullanarak turtle1/cmd_vel topici üzerinden hız verisi yayınlamaktadır bu sayede kaplumbağanın ileri hareket etmesini ve dönmesini sağlayabildim gönderilen Twist mesajlarının turtlesim tarafından doğrudan algılandığını ve kaplumbağanın bu komutlara göre hareket ettiğini gözlemledim yazdığım node u colcon ile derledim ve çalışma alanını source ettikten sonra ros2 run komutu ile çalıştırdım node çalıştığında kaplumbağanın ekranda hareket etmesi topic yapısının doğru çalıştığını gösterdi

```
omer@omer-Sword-16-HX-B14VGKG:~$ cd ~/ros2_ws
source install/setup.bash
ros2 run omer_deneme cmd_vel_publisher
[INFO] [1765707461.601565161] [cmd_vel_publisher]: Twist mesajı gönderildi
[INFO] [1765707462.081043614] [cmd_vel_publisher]: Twist mesajı gönderildi
[INFO] [1765707462.580281736] [cmd_vel_publisher]: Twist mesajı gönderildi
[INFO] [1765707463.080422389] [cmd_vel_publisher]: Twist mesajı gönderildi
[INFO] [1765707463.580444466] [cmd_vel_publisher]: Twist mesajı gönderildi
[INFO] [1765707464.080187474] [cmd_vel_publisher]: Twist mesajı gönderildi
[INFO] [1765707464.580173278] [cmd_vel_publisher]: Twist mesajı gönderildi
□
```



```
omer@omer-Sword-16-HX-B14VGKG:~$ cd ~/ros2_ws/src/omer_deneme/omer_deneme
omer@omer-Sword-16-HX-B14VGKG:~/ros2_ws/src/omer_deneme/omer_deneme$ ls
__init__.py
omer@omer-Sword-16-HX-B14VGKG:~/ros2_ws/src/omer_deneme/omer_deneme$ nano cmd_vel_publisher.py
omer@omer-Sword-16-HX-B14VGKG:~/ros2_ws/src/omer_deneme/omer_deneme$ cd ~/ros2_ws/src/omer_deneme/omer_deneme
nano cmd_vel_publisher.py
omer@omer-Sword-16-HX-B14VGKG:~/ros2_ws/src/omer_deneme/omer_deneme$ chmod +x cmd_vel_publisher.py
omer@omer-Sword-16-HX-B14VGKG:~/ros2_ws/src/omer_deneme/omer_deneme$ nano ~/ros2_ws/src/omer_deneme/setup.py
omer@omer-Sword-16-HX-B14VGKG:~/ros2_ws/src/omer_deneme/omer_deneme$ cd ~/ros2_ws
colcon build
Starting >>> omer_deneme
Finished <<< omer_deneme [0.63s]

Summary: 1 package finished [0.71s]
omer@omer-Sword-16-HX-B14VGKG:~/ros2_ws$ source install/setup.bash
omer@omer-Sword-16-HX-B14VGKG:~/ros2_ws$ ros2 pkg list | grep omer
omer_deneme
omer@omer-Sword-16-HX-B14VGKG:~/ros2_ws$ □
```

```

GNU nano 7.2                                     cmd_vel_publisher.py *
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist

class CmdVelPublisher(Node):

    def __init__(self):
        super().__init__('cmd_vel_publisher')

        self.publisher_ = self.create_publisher(
            Twist,
            '/turtle1/cmd_vel',
            10
        )

        self.timer = self.create_timer(0.5, self.timer_callback)

    def timer_callback(self):
        msg = Twist()
        msg.linear.x = 2.0
        msg.angular.z = 1.0

        self.publisher_.publish(msg)
        self.get_logger().info('Twist mesajı gönderildi')

def main(args=None):
    rclpy.init(args=args)

    node = CmdVelPublisher()
    rclpy.spin(node)

    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

```

omer@omer-Sword-16-HX-B14VGKG:~$ mkdir -p ~/ros2_ws/src
omer@omer-Sword-16-HX-B14VGKG:~$ cd ~/ros2_ws
ls
src
omer@omer-Sword-16-HX-B14VGKG:~/ros2_ws$ 

```

```

omer@omer-Sword-16-HX-B14VGKG:~$ cd ~/ros2_ws
ls
build install log src
omer@omer-Sword-16-HX-B14VGKG:~/ros2_ws$ cd ~/ros2_ws/src
tree omer_deneme
omer_deneme
├── omer_deneme
│   ├── cmd_vel_publisher.py
│   └── __init__.py
├── package.xml
├── resource
│   └── omer_deneme
├── setup.cfg
├── setup.py
└── test
    ├── test_copyright.py
    ├── test_flake8.py
    └── test_pep257.py

4 directories, 9 files
omer@omer-Sword-16-HX-B14VGKG:~/ros2_ws/src$ cd ~/ros2_ws
colcon build
Starting >>> omer_deneme
Finished <<< omer_deneme [0.66s]

Summary: 1 package finished [0.72s]

```


UNİTY KURAMADIM